Министерство образования Республики Беларусь

Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра Информатики
Дисциплина «Информационные сети. Основы безопасности»

ОТЧЁТ
к лабораторной работе №6
на тему
«ЗАЩИТА ОТ АТАКИ МЕТОДОМ ВНЕДРЕНИЯ SQL-КОДА»
БГУИР 6-05-0612-02 23

Выполнил студент группы 353503
СЕБЕЛЕВ Дмитрий Юрьевич

_____
(дата, подпись студента)

Проверил ассистент каф. информатики
ГЕРЧИК Артем Вадимович

_____
(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

1 Постановка задачи 3
2 Выполнение работы 4
2.1 Теоретические сведения 4
2.2 Уязвимая и безопасная аутентификация 4
2.3 Уязвимый и безопасный поиск 5
2.4 Уязвимое и безопасное удаление 6
2.5 Результаты тестирования 6
Вывод 8
Приложение А (обязательное) Листинг программного кода 9

1 ПОСТАНОВКА ЗАДАЧИ

Целью данной лабораторной работы является изучение механизмов атаки «внедрение SQL-кода» (*SQL Injection*) и методов защиты от неё. В рамках работы необходимо разработать приложение, взаимодействующее с базой данных, которое демонстрирует эксплуатацию уязвимостей в различных SQL-запросах, а также безопасные способы работы с данными с использованием параметризованных запросов.

2 ВЫПОЛНЕНИЕ РАБОТЫ

2.1 Теоретические сведения

Внедрение SQL-кода (*SQL Injection*) – это один из наиболее распространённых и опасных видов уязвимостей приложений. Эта атака возникает, когда пользовательские данные без должной проверки и экранирования напрямую конкатенируются со строкой SQL-запроса.

В результате злоумышленник получает возможность внедрить собственные SQL-команды, которые будут выполнены сервером базы данных в контексте приложения. Эксплуатация данной уязвимости может привести к обходу механизмов аутентификации, чтению, изменению или удалению конфиденциальных данных, а в некоторых случаях – к выполнению команд на сервере баз данных.

Для демонстрации уязвимости и защиты от неё была разработана база данных с таблицей *users*, содержащей поля *id*, *username*, *password* и *role*.

2.2 Уязвимая и безопасная аутентификация

Функция *vulnerableLogin* демонстрирует типичную ошибку при реализации механизма аутентификации – формирование SQL-запроса путём прямой конкатенации строк:

Листинг 2.1 – Уязвимый SQL-запрос аутентификации

```
query := fmt.Sprintf("SELECT id, username, password, role FROM users
WHERE username='%s' AND password='%s'", username, password)
```

При вводе пользователем в качестве логина и пароля строки *admin' OR '1'='1'--* сформированный запрос принимает следующий вид:

Листинг 2.2 – Вид запроса после внедрения

```
SELECT id, username, password, role FROM users WHERE username='admin'
OR '1'='1'-- AND password='admin'
```

Поскольку условие *'1'='1'* всегда истинно, база данных возвращает первую запись из таблицы и приложение успешно аутентифицирует пользователя без знания реального пароля.

Для защиты от подобных атак в функции *secureLogin* применяются параметризованные запросы:

Листинг 2.3 – Безопасный SQL-запрос аутентификации

```
query := "SELECT id, username, password, role FROM users WHERE
username=$1 AND password=$2"
err := s.db.QueryRow(context.Background(), query, username,
password).Scan(&id, &dbUsername, &dbPassword, &role)
```

При таком подходе база данных компилирует структуру SQL-запроса до подстановки параметров. Пользовательский ввод трактуется базой данных исключительно как данные, а не как исполняемый код, что делает невозможным изменение логики запроса.

2.3 Уязвимый и безопасный поиск

В функции *vulnerableSearchUser* реализован поиск по частичному совпадению имени пользователя с использованием оператора *LIKE*.

Листинг 2.4 – Уязвимый SQL-запрос поиска

```
query := fmt.Sprintf("select username, role from users where username
like '%%%s%%'", searchTerm)
```

Данная конструкция уязвима к атаке типа *UNION-based SQL Injection*. Злоумышленник может передать строку вида *admin' UNION SELECT password, role FROM users*--, что приведёт к формированию следующего запроса:

Листинг 2.5 – Запрос с использованием UNION

```
select username, role from users where username like '%admin' UNION
SELECT password, role FROM users--%'
```

Оператор *UNION* объединяет результаты двух запросов, позволяя извлечь скрытые данные (например, пароли пользователей). Двойной дефис `--` используется для комментирования оставшейся части исходного запроса.

Для обеспечения безопасности в функции *secureUserSearch* применяются параметризованные запросы с предварительным форматированием строки поиска на уровне приложения:

Листинг 2.6 – Безопасный SQL-запрос поиска

```
searchTerm = "%" + strings.TrimSpace(searchTerm) + "%"
query := "select username, role from users where username like $1"
rows, err := s.db.Query(context.Background(), query, searchTerm)
```

2.4 Уязвимое и безопасное удаление

Функция *vulnarableDeleteUser* удаляет пользователя по его имени, вновь используя конкатенацию строк. Это открывает возможность для атаки с использованием множественных запросов (*Stacked Queries*):

Листинг 2.7 – Запрос, уязвимый к Stacked Queries

```
delete from users where username=''; DELETE FROM users;--'
```

Вводя строку *'; DELETE FROM users;--*, злоумышленник завершает текущий запрос и начинает новый, который удаляет все записи из таблицы.

Защита в функции *secureDeleteUser* также строится на базе параметризованных запросов:

Листинг 2.8 – Безопасный запрос удаления

```
query := "delete from users where username=$1"
commandTag, err := s.db.Exec(context.Background(), query, username)
```

2.5 Результаты тестирования

Для проверки успешности атак и надёжности защитных механизмов было проведено тестирование всех описанных функций.

В первом тесте была произведена попытка обхода аутентификации с использованием конструкции *' OR '1'='1'*. Как видно на рисунке 2.1, атака завершилась успехом: приложение авторизовало пользователя как администратора и вывело его данные, включая пароль. Использование параметризованных запросов в защищённой версии предотвратило эксплуатацию данной уязвимости.

```
--- Vulnerable Login ---
Enter username: idk' or '1'='1'--
Enter password: ajllasdkjf
[DEBUG] Query: SELECT id, username, password, role FROM users WHERE username='idk' or '1'='1'--' AND password='ajllasdkjf'
[SUCCESS] User authenticated!
  ID: 1
  Username: admin
  Role: admin
  [WARNING] SQL Injection detected! You logged in as admin without knowing their password.
  Stolen Password: super-secret-pass
```

Рисунок 2.1 – Обход аутентификации через внедрение SQL-кода

Второй тест демонстрирует извлечение паролей из базы данных с помощью оператора *UNION* (рисунок 2.2). Злоумышленник смог получить список всех пользователей и их паролей, которые не выводились при обычном поиске.

```
--- Vulnerable Search ---
Enter search term: a' union select username, password from users--
[DEBUG] Query: select username, role from users where username like '%a' union select username, password from users--%'
bob, password
alice, qwerty123
admin, super-secret-pass
```

Рисунок 2.2 – Извлечение конфиденциальных данных с использованием UNION

Третий тест иллюстрирует возможность выполнения нескольких запросов подряд. Ввод полезной нагрузки *'; DELETE FROM users;--* привёл к выполнению второй команды *DELETE*, что повлекло за собой удаление всех записей из базы данных (рисунок 2.3).

```
--- Vulnerable User Delete ---
Enter username: alice'; drop table users--
[DEBUG] Query: delete from users where username='alice'; drop table users--'
[WARNING] No rows were affected

========== SQL Injection Demo ==========
1. Vulnerable login (or '1'='1')
2. Secure login (parameterized queries)
3. Vulnerable user search (union injection)
4. Secure user search (parameterized queries)
5. Vulnerable user delete (stacked queries)
6. Secure user delete (parameterized queries)
7. Exit

Choose option: 4

--- Secure user search ---
Enter search term: a
[DEBUG] Query: select username, role from users where username like $1
[DEBUG] Parameters: search term="%a%"
[FAIL] Search failed: ERROR: relation "users" does not exist (SQLSTATE 42P01)
```

Рисунок 2.3 – Удаление данных с использованием Stacked Queries

ВЫВОД

В ходе выполнения лабораторной работы были изучены механизмы возникновения уязвимости «внедрение SQL-кода» (*SQL Injection*) и методы защиты от неё.

Было разработано приложение, демонстрирующее три основных вектора атак: обход аутентификации с использованием тавтологий (*' OR '1'='1'*), извлечение данных из других таблиц с помощью объединения запросов (*UNION-based SQL Injection*) и выполнение деструктивных команд через объединение нескольких запросов (*Stacked Queries*).

Анализ уязвимого кода показал, что причиной успешности атак является конкатенация пользовательского ввода и текста SQL-запроса, что позволяет злоумышленнику изменять логику его выполнения.

Для каждого уязвимого сценария был реализован безопасный аналог. В качестве основного метода защиты применялись параметризованные запросы (*Prepared Statements*), которые разделяют данные и код запроса на уровне СУБД. Тестирование подтвердило, что данный подход полностью предотвращает внедрение SQL-кода, так как пользовательский ввод воспринимается исключительно как данные и не может быть выполнен как команда.

ПРИЛОЖЕНИЕ А
(обязательное)
Листинг программного кода

Листинг А.1 – Код инициализации базы данных

```
create table users (
    id serial PRIMARY KEY,
    username text not null,
    password text not null,
    role text not null
);

INSERT INTO users (username, password, role) VALUES
('admin', 'super-secret-pass', 'admin'),
('alice', 'qwerty123', 'user'),
('bob', 'password', 'user');
```

Листинг А.2 – Код программы

```
package main

import (
  "bufio"
  "context"
  "fmt"
  "os"
  "strings"

  "github.com/jackc/pgx/v5"
  "github.com/joho/godotenv"
)

type service struct {
  db *pgx.Conn
}

func main() {
  godotenv.Overload()

           conn,     err     :=     pgx.Connect(context.Background(),
os.Getenv("DATABASE_URL"))
  if err != nil {
    fmt.Fprintf(os.Stderr, "Unable to connect to database: %v\n", err)
    os.Exit(1)
  }
  defer conn.Close(context.Background())

  serv := service{
    db: conn,
  }

  serv.runMenu()
}
```

```
func (s *service) runMenu() {
  r := bufio.NewReader(os.Stdin)

  for {
    fmt.Println("\n========== SQL Injection Demo ==========")
    fmt.Println("1. Vulnerable login (or '1'='1')")
    fmt.Println("2. Secure login (parameterized queries)")
    fmt.Println("3. Vulnerable user search (union injection)")
    fmt.Println("4. Secure user search (parameterized queries)")
    fmt.Println("5. Vulnerable user delete (stacked queries)")
    fmt.Println("6. Secure user delete (parameterized queries)")
    fmt.Println("7. Exit")
    fmt.Print("\nChoose option: ")

    choice, _ := r.ReadString('\n')
    choice = strings.TrimSpace(choice)

    switch choice {
    case "1":
      s.vulnerableLogin(r)
    case "2":
      s.secureLogin(r)
    case "3":
      s.vulnerableSearchUser(r)
    case "4":
      s.secureUserSearch(r)
    case "5":
      s.vulnarableDeleteUser(r)
    case "6":
      s.secureDeleteUser(r)
    case "7":
      fmt.Println("Goodbye!")
      os.Exit(0)
    default:
      fmt.Println("Invalid option, try again")
    }
  }
}

func (s *service) vulnerableSearchUser(r *bufio.Reader) {
  fmt.Println("\n--- Vulnerable Search ---")
  fmt.Print("Enter search term: ")

  searchTerm, _ := r.ReadString('\n')
  searchTerm = strings.TrimSpace(searchTerm)

  query := fmt.Sprintf("select username, role from users where username
like '%%%s%%'", searchTerm)
  fmt.Printf("[DEBUG] Query: %s\n", query)

  rows, err := s.db.Query(context.Background(), query)
  if err != nil {
    fmt.Printf("[FAIL] Search failed: %v\n", err)
    return
  }
  defer rows.Close()
```

```
  for rows.Next() {
    var username string
    var role string
    err := rows.Scan(&username, &role)
    if err != nil {
      fmt.Printf("[FAIL] Unable to scan row: %v\n", err)
      return
    }
    fmt.Printf("%s, %s\n", username, role)
  }
}

func (s *service) secureUserSearch(r *bufio.Reader) {
  fmt.Println("\n--- Secure user search ---")
  fmt.Print("Enter search term: ")
  searchTerm, _ := r.ReadString('\n')
  searchTerm = "%" + strings.TrimSpace(searchTerm) + "%"

  query := "select username, role from users where username like $1"
  fmt.Printf("[DEBUG] Query: %s\n", query)
  fmt.Printf("[DEBUG] Parameters: search term=%q\n", searchTerm)

  rows, err := s.db.Query(context.Background(), query, searchTerm)
  if err != nil {
    fmt.Printf("[FAIL] Search failed: %v\n", err)
    return
  }

  for rows.Next() {
    var username string
    var role string
    err := rows.Scan(&username, &role)
    if err != nil {
      fmt.Printf("[FAIL] Unable to scan row: %v\n", err)
      return
    }
    fmt.Printf("%s, %s\n", username, role)
  }
}

func (s *service) vulnerableLogin(r *bufio.Reader) {
  fmt.Println("\n--- Vulnerable Login ---")
  fmt.Print("Enter username: ")
  username, _ := r.ReadString('\n')
  username = strings.TrimSpace(username)

  fmt.Print("Enter password: ")
  password, _ := r.ReadString('\n')
  password = strings.TrimSpace(password)

  query := fmt.Sprintf("SELECT id, username, password, role FROM users
WHERE username='%s' AND password='%s'", username, password)
  fmt.Printf("[DEBUG] Query: %s\n", query)

  var id int
  var dbUsername, dbPassword, role string
```

```
      err   :=   s.db.QueryRow(context.Background(),   query).Scan(&id,
&dbUsername, &dbPassword, &role)
  if err != nil {
    fmt.Printf("[FAIL] Authentication failed: %v\n", err)
    return
  }

  fmt.Printf("[SUCCESS] User authenticated!\n")
  fmt.Printf("  ID: %d\n", id)
  fmt.Printf("  Username: %s\n", dbUsername)
  fmt.Printf("  Role: %s\n", role)

  if password != dbPassword {
    fmt.Printf("  [WARNING] SQL Injection detected! You logged in as %s
without knowing their password.\n", dbUsername)
    fmt.Printf("  Stolen Password: %s\n", dbPassword)
  }
}

func (s *service) secureLogin(r *bufio.Reader) {
  fmt.Println("\n--- Secure Login ---")
  fmt.Print("Enter username: ")
  username, _ := r.ReadString('\n')
  username = strings.TrimSpace(username)

  fmt.Print("Enter password: ")
  password, _ := r.ReadString('\n')
  password = strings.TrimSpace(password)

  fmt.Println("\n[SECURE] Executing parameterized query...")

   query  :=  "SELECT  id,  username,  password,  role  FROM  users  WHERE
username=$1 AND password=$2"
  fmt.Printf("[DEBUG] Query: %s\n", query)
  fmt.Printf("[DEBUG] Parameters: username=%q, password=%q\n", username,
password)

  var id int
  var dbUsername, dbPassword, role string

     err   :=   s.db.QueryRow(context.Background(),   query,   username,
password).Scan(&id, &dbUsername, &dbPassword, &role)
  if err != nil {
    fmt.Printf("[FAIL] Authentication failed: %v\n", err)
    return
  }

  fmt.Printf("[SUCCESS] User authenticated!\n")
  fmt.Printf("  ID: %d\n", id)
  fmt.Printf("  Username: %s\n", dbUsername)
  fmt.Printf("  Role: %s\n", role)
}

func (s *service) vulnarableDeleteUser(r *bufio.Reader) {
  fmt.Println("\n--- Vulnerable User Delete ---")
  fmt.Print("Enter username: ")
```

```
  username, _ := r.ReadString('\n')
  username = strings.TrimSpace(username)

  query := fmt.Sprintf("delete from users where username='%s'", username)
  fmt.Printf("[DEBUG] Query: %s\n", query)

  commandTag, err := s.db.Exec(context.Background(), query)
  if err != nil {
    fmt.Printf("[FAIL] Deletion failed: %v\n", err)
    return
  }

  if commandTag.RowsAffected() == 0 {
    fmt.Println("[WARNING] No rows were affected")
  } else {
    fmt.Printf("User successfully deleted")
  }
}

func (s *service) secureDeleteUser(r *bufio.Reader) {
  fmt.Println("\n--- Secure User Delete---")
  fmt.Print("Enter username: ")
  username, _ := r.ReadString('\n')
  username = strings.TrimSpace(username)

  query := "delete from users where username=$1"

  fmt.Printf("[DEBUG] Query: %s\n", query)
  fmt.Printf("[DEBUG] Parameters: username=%s\n", username)

  commandTag, err := s.db.Exec(context.Background(), query, username)
  if err != nil {
    fmt.Printf("[FAIL] Deletion failed: %v\n", err)
    return
  }

  if commandTag.RowsAffected() == 0 {
    fmt.Printf("[WARNING] No rows were affected: %v\n", err)
  } else {
    fmt.Printf("User successfully deleted")
  }
}
```